SHERLOCK

Security Review For veil.cash

Collaborative Audit Prepared For:	**veil.cash**
Lead Security Expert(s):	**1nc0gn170** **blockdev**
Date Audited:	**March 2 - March 6, 2026**

Introduction

Veil Cash lets trusted users send and receive crypto privately on Base, using ZK-SNARKs under the hood.

Scope

Repository: veildotcash/veil_pools_audit

Audited Commit: 60e90b151a98731c27f8aa265db1244380be5295

Final Commit: 60e90b151a98731c27f8aa265db1244380be5295

Files:

- docs/architecture.md
- docs/CONVENTIONS.md
- docs/FLOWS.md
- docs/OVERVIEW.md
- src/lib/VeilLib.sol
- src/VeilPool/VeilETHPool.sol
- src/VeilPool/VeilUSDCPool.sol
- src/VeilPool/verifiers/MerkleTreeWithHistory.sol
- src/VeilPool/verifiers/Verifier16.sol
- src/VeilPool/verifiers/Verifier2.sol

Final Commit Hash

60e90b151a98731c27f8aa265db1244380be5295

Findings

Each issue has an assigned severity:

- High issues are directly exploitable security vulnerabilities that need to be fixed.
- Medium issues are security vulnerabilities that may not be directly exploitable or may require certain conditions in order to be exploited. All major issues should be addressed.
- Low/Info issues are non-exploitable, informational findings that do not pose a security risk or impact the system's integrity. These issues are typically cosmetic or related to compliance requirements, and are not considered a priority for remediation.

Issues Found

High	Medium	Low/Info
0	0	4

Issues Not Fixed and Not Acknowledged

High	Medium	Low/Info
0	0	0

Issue L-1: Deployer shouldn't be assigned the ownership role [ACKNOWLEDGED]

Source: https://github.com/sherlock-audit/2026-02-veil-cash-mar-2nd-2026/issues/11

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

At deploy time, deployer is made the owner.

Vulnerability Detail

Assigning deployer an admin role isn't a recommended practice as deployer is an EOA. Even if the role is later transferred to a multi-sig, this temporary role assignment can be lead to issues if the private key is known to an attacker.

Impact

Existing leak of the private key can lead to a third party taking over the role maliciously.

Code Snippet

```
constructor(IVerifier _verifier2, IVerifier _verifier16, uint32 _levels, address
↪  _hasher, IERC20 _usdc)
    Ownable(msg.sender)
```

Tool Used

Manual Review

Recommendation

Take an explicit `address _owner` argument in constructor for the ownership role.

Discussion

apexethdev

Noted.

Issue L-2: Documentation incorrectly states that pools are only called by Entry contract [ACKNOWLEDGED]

Source: https://github.com/sherlock-audit/2026-02-veil-cash-mar-2nd-2026/issues/12

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

`architecture.md` states that pools are called only by the validated Entry contract via `onlyValidator`, but that is only true for deposit functions. Withdrawal and transfer functions are public and permissionless.

Vulnerability Detail

`architecture.md` states that pools are called only by the validated Entry contract via `onlyValidator`, but that is only true for deposit functions. Withdrawal and transfer functions are public and permissionless.

Impact

Documentation misalignment.

Code Snippet

```
Pools are called only by the validated Entry contract (`onlyValidator`).
```

Tool Used

Manual Review

Recommendation

Update docs to say deposit functions are only called by the validated Entry contract.

Discussion

apexethdev

Noted - will update.

Issue L-3: VeilETHPool doesn't emit a withdraw event [ACKNOWLEDGED]

Source: https://github.com/sherlock-audit/2026-02-veil-cash-mar-2nd-2026/issues/13

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

VeilETHPool doesn't emit a withdrawal event.

Vulnerability Detail

VeilUSDCPool version emits `USDCWithdrawal` on withdraw, but the VeilETHPool doesn't.

Impact

Having this event can make indexing easier.

Code Snippet

```
function withdrawETH(VeilLib.Proof memory _args, VeilLib.ExtData memory _extData)
↪  public {
    // Only allow withdrawals
    if (_extData.extAmount >= 0) revert VeilLib.OnlyForWithdrawals();

    _transact(_args, _extData);
}
```

Tool Used

Manual Review

Recommendation

Add an ETH withdrawal event.

Discussion

apexethdev

Noted. Aware of this one, we added this event to the USDC pool.

Issue L-4: Unused Constants [ACKNOWLEDGED]

Source: https://github.com/sherlock-audit/2026-02-veil-cash-mar-2nd-2026/issues/14

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

VeilLib defines `BASIS_POINTS` and MerkleTreeWithHistory defines `ZERO_VALUE`, both are never referenced.

Vulnerability Detail

`VeilLib.BASIS_POINTS` is declared but never used by any in-scope contract.

`MerkleTreeWithHistory.ZERO_VALUE` is declared but never referenced – the contract uses the `zeros(0)` function to get the same value instead.

Impact

Dead code increases contract size and deployment gas cost.

Code Snippet

```
// src/lib/VeilLib.sol:55
uint256 public constant BASIS_POINTS = 10000;

// src/VeilPool/verifiers/MerkleTreeWithHistory.sol:14
uint256 public constant ZERO_VALUE =
↪  21663839004416932945382355908790599225266501822907911457504978515578255421292;
```

Tool Used

Manual Review

Recommendation

Remove both unused constants.

Discussion

apexethdev

Noted.

Disclaimers

Sherlock does not provide guarantees nor warranties relating to the security of the project.

Usage of all smart contract software is at the respective users' sole risk and is the users' responsibility.